

# NiftyDrum

---

## Official Documentation

Ronna Technologies

Copyright © 2025 Ronna Technologies

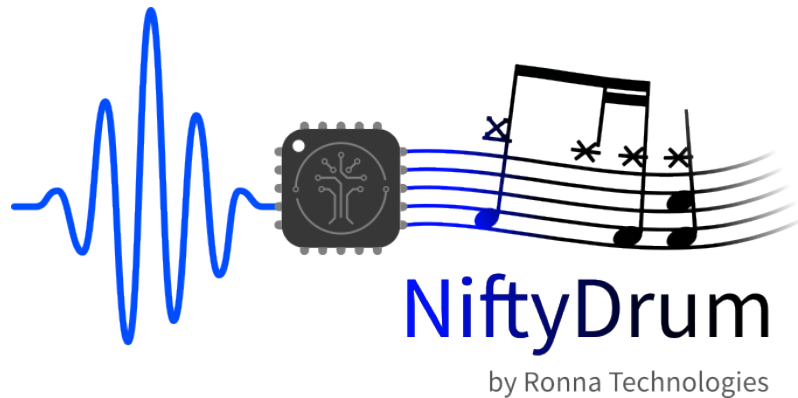
# Table of contents

---

|                                  |    |
|----------------------------------|----|
| 1. About NiftyDrum               | 3  |
| 1.1 Description                  | 3  |
| 1.2 How It Works                 | 3  |
| 1.3 Specifications               | 4  |
| 2. The Board                     | 5  |
| 2.1 Description                  | 5  |
| 2.2 Connecting Sensors           | 6  |
| 2.3 MIDI Outputs                 | 6  |
| 2.4 USB-C Port                   | 7  |
| 2.5 Raspberry-Pi hat form factor | 7  |
| 2.6 App Features                 | 7  |
| 3. The App                       | 8  |
| 3.1 Configure NiftyDrum          | 8  |
| 3.2 How to Install the App       | 10 |
| 3.3 Piezo Trigger Configuration  | 11 |
| 3.4 Hi-Hat Pedal Configuration   | 12 |
| 3.5 Firmware Upgrade             | 13 |
| 4. Serial Protocol               | 16 |
| 4.1 Overview                     | 16 |
| 4.2 Command Syntax               | 16 |
| 4.3 Hi-Hat Controller Parameters | 19 |
| 5. Arduino                       | 21 |
| 5.1 That's a Blue Pill           | 21 |
| 5.2 Arduino IDE                  | 21 |

# 1. About NiftyDrum

---



## 1.1 Description

---

NiftyDrum is a trigger-to-MIDI conversion module that transforms piezo and FSR sensor inputs into MIDI messages. Connect up to 9 piezo sensors and 1 FSR (Force Sensing Resistor) to the dedicated terminal blocks, then receive MIDI data via USB-C connection.

## 1.2 How It Works

---

NiftyDrum delivers high-level MIDI performance in 4 easy steps:

- **Connect sensors:** Attach up to 9 piezo sensors and 1 FSR to the terminal blocks
- **Plug in:** Connect to your DAW, Raspberry Pi, or drum module via USB
- **Configure:** Use the web-based GUI to adjust trigger parameters, MIDI mapping, and velocity curves
- **Play:** Notes are transmitted instantly with imperceptible latency

## 1.3 Specifications

---

### 1.3.1 Hardware

---

- **Piezo inputs:** 9 channels
- **FSR input:** 1 channel (hi-hat controller)
- **Connector type:** Terminal blocks
- **USB interface:** Type-C
- **Dimensions:** 65 × 56.5 mm

### 1.3.2 Performance

---

- **Latency:** <2.5 ms
- **Sample rate:** >10 kHz
- **Velocity resolution:** 127 levels (full MIDI range)

### 1.3.3 Software

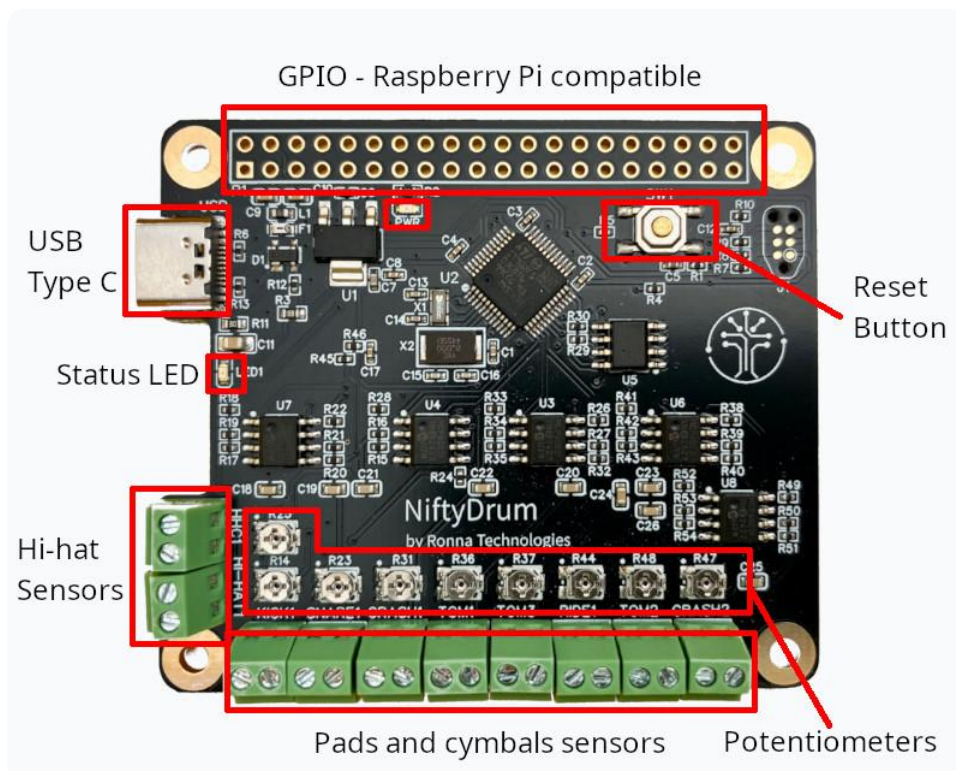
---

- **Platform support:** Windows, macOS, Linux
- **User interface:** Web-based application
- **Firmware updates:** Via USB
- **MIDI output:** Note messages and Control Changes (CC)

## 2. The Board

### 2.1 Description

The NiftyDrum board is shown in the following image.



This board features the following interfaces:

- Terminal blocks for sensor inputs
- USB Type-C port for laptop or PC connectivity
- 9 potentiometers for sensitivity adjustment
- 2 LEDs
- Reset button
- Raspberry Pi-compatible GPIO header
- 4 mounting holes

## 2.2 Connecting Sensors

---

The board provides 9 piezo inputs, supporting up to 9 single-zone pads, as well as 1 FSR input.

### 2.2.1 Hi-Hat Sensors

---

On the left side of the board, two dedicated terminal blocks are reserved for hi-hat sensors:

- **Top terminal block:** Connects to an FSR (Force Sensing Resistor) sensor for hi-hat controller input
- **Bottom terminal block:** Connects to a piezo sensor for hi-hat cymbal trigger

### 2.2.2 Standard Pads and Cymbals

---

The remaining eight terminal blocks, located at the bottom of the board, are for connecting regular pads and cymbals. While the board labels indicate the default firmware assignments, these inputs are fully customizable.

### 2.2.3 Important Notes

---

- For all terminal blocks, the ground pin is positioned on the **left-hand side**
- Nine onboard potentiometers enable hardware-level sensitivity adjustments for maximum flexibility
- If unsure about sensitivity settings, leave potentiometers at their midpoint for balanced performance

## 2.3 MIDI Outputs

---

The board offers two methods for transmitting MIDI notes and Control Changes:

- **USB-C port:** Outputs USB MIDI messages
- **GPIO UART pins:** Raspberry Pi GPIO-compatible interface

## 2.4 USB-C Port

---

Using NiftyDrum as a USB device is the recommended method for receiving MIDI messages. This configuration enables:

- Integration with DAW software for high-quality sound output from your laptop
- Control and configuration via the [official app](#)

## 2.5 Raspberry-Pi hat form factor

---

The board is designed with a Raspberry Pi 4 HAT form factor, ensuring seamless integration.

## 2.6 App Features

---

The official app provides comprehensive control over your NiftyDrum board:

- Customize MIDI note assignments for each trigger
- Design custom velocity curves per trigger
- Adjust advanced parameters including gain, threshold, scan time, mask time, and decay
- Update board firmware to the latest version

## 3. The App

---

### 3.1 Configure NiftyDrum

---

#### Warning

This app is designed **only for configuration purposes**. While running, it continuously transfers data between your PC and the board. For optimal performance, **close the app before playing**.

NiftyDrum is fully configurable, allowing you to adjust parameters like scan time, mask time, decay, threshold, etc. To simplify customization, a dedicated desktop application is available, compatible with Windows, Linux, and macOS.

Below are all the different commands the app can send to the board.

#### 3.1.1 General Board Commands

---

| Command                 | Description                               |
|-------------------------|-------------------------------------------|
| Reset                   | Restart the board in bootloader mode      |
| Serial number           | Retrieve the board's unique serial number |
| Version                 | Retrieve the current firmware version     |
| Save current parameters | Persist current settings to the board     |
| Load parameters         | Load previously saved board parameters    |
| Factory reset           | Reset all parameters to factory defaults  |



## 3.1.2 Trigger Parameters (Per Trigger, Including Hi-Hat Cymbal)

---

| Parameter              | Description                                    |
|------------------------|------------------------------------------------|
| Set/get velocity curve | Adjust or retrieve the velocity response curve |
| Set/get threshold      | Configure the trigger activation threshold     |
| Scan time              | Set/get the trigger scan time                  |
| Mask time              | Set or adjust the trigger mask time            |
| Decay time             | Adjust the decay time of the trigger           |
| Gain                   | Adjust the gain level of the trigger           |
| MIDI Note              | Assign the MIDI note for the trigger           |

## 3.1.3 Hi-Hat Pedal Parameters

---

| Parameter          | Description                               |
|--------------------|-------------------------------------------|
| Update interval    | Set the hi-hat pedal update frequency     |
| Noise threshold    | Ignore pedal changes below this value     |
| Pedal offset       | Determine if the hi-hat is fully closed   |
| Velocity threshold | Set the velocity threshold for foot chick |

## 3.1.4 How the App Works

---

The app simplifies customization by organizing everything logically: instruments are selected via a drop-down, while MIDI notes and velocity curves are managed separately from trigger settings for a cleaner, more efficient setup.

## 3.2 How to Install the App

---

The app is available for Windows, macOS, and Linux and can be downloaded directly from the official [NiftyDrum.com](https://niftydrum.com) website. Follow the OS-specific instructions provided on the site.

### 3.2.1 Windows

---

On Windows, the app is distributed as a `.zip` file, so no installation is required, simply extract and run it. Note that, if that's not already done, you will have to install the [Microsoft Visual C++ Redistributable package](#).

### 3.2.2 Linux

---

For Linux, the app is packaged as a `.deb` file. You can install it using your preferred package manager or by running the following command in a terminal:

Ubuntu 22.04

Ubuntu 24.04

debian

```
sudo apt install --reinstall ./NiftyDrum-1.0.0-Ubuntu-22.04.deb
```

```
sudo apt install --reinstall ./NiftyDrum-1.0.0-Ubuntu-24.04.deb
```

```
sudo apt install --reinstall ./NiftyDrum-1.0.0-Linux.deb
```

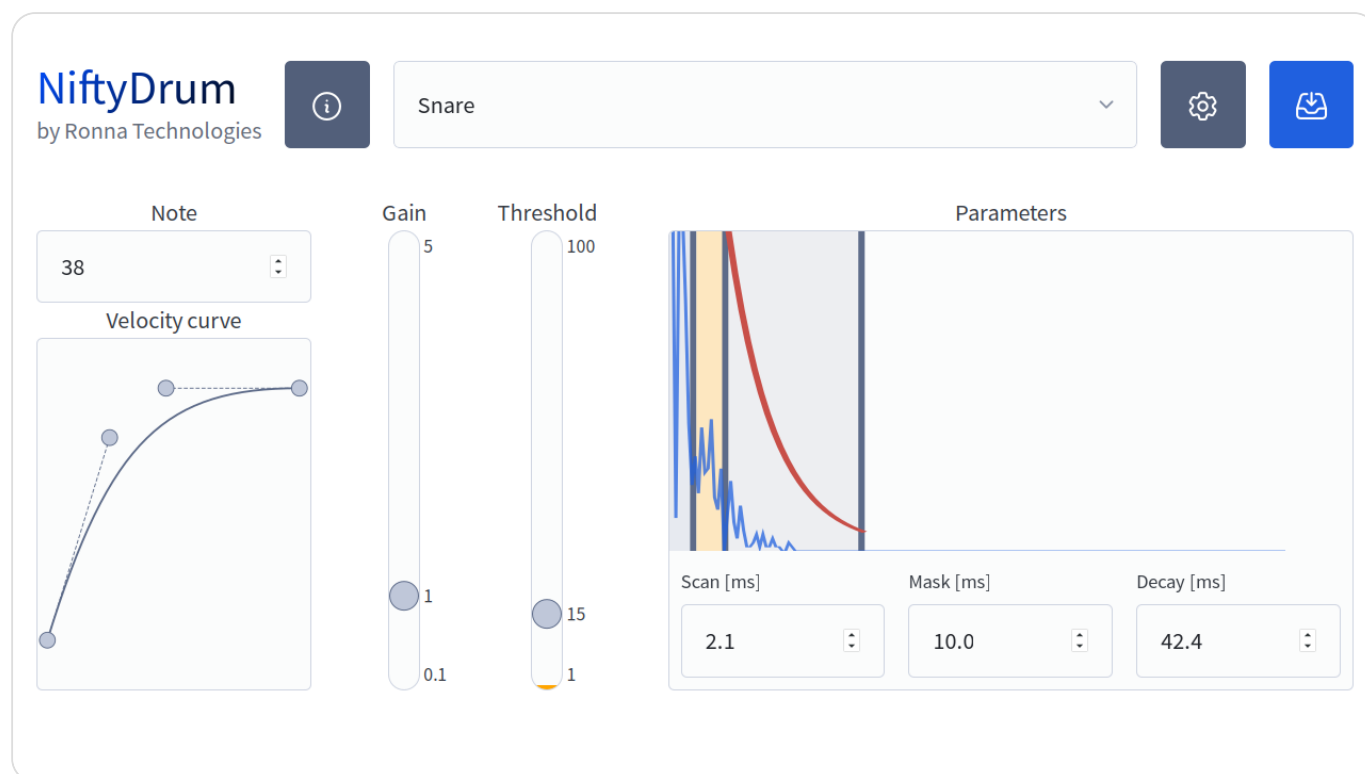
### 3.2.3 MacOS

---

The MacOS version of the app is provided as a `.zip` file. Just extract it and run the application. If you're using an Apple Silicon Mac, you may be prompted to install Rosetta the first time you launch the app.

## 3.3 Piezo Trigger Configuration

Customize each piezo trigger by selecting your desired instrument from the dropdown menu. The screenshot below illustrates the process for configuring the snare drum.



On the left side, you can assign the MIDI note for the pad, such as note 38 for the snare. Below the note input, you'll find the velocity curve editor, which uses Bézier controls for precise adjustments. You can drag and drop the endpoints and the two middle control points to shape the curve according to your needs. The horizontal axis reflects the raw MIDI velocity detected by the sensor, while the vertical axis shows the velocity value transmitted over USB. This setup lets you fine-tune the responsiveness and dynamics of your triggers.

Moving from left to right, you can adjust the trigger gain within a range of 0.1 to 5. This allows you to boost the input sensitivity of the trigger, enhancing the volume of ghost notes when the velocity curve alone isn't sufficient. It's important to note that the gain is applied after a strike has been detected, so it doesn't impact the threshold setting.

Next is the threshold setting, which sets the minimum signal level the piezo must exceed for a hit to register. This ensures only intentional strikes are detected, effectively filtering out unwanted noise and preventing false triggers.

## 3.4 Hi-Hat Pedal Configuration

The last item in the dropdown menu is the **hi-hat pedal configuration**. Unlike other inputs, it does not include standard parameters like scan time or mask time.

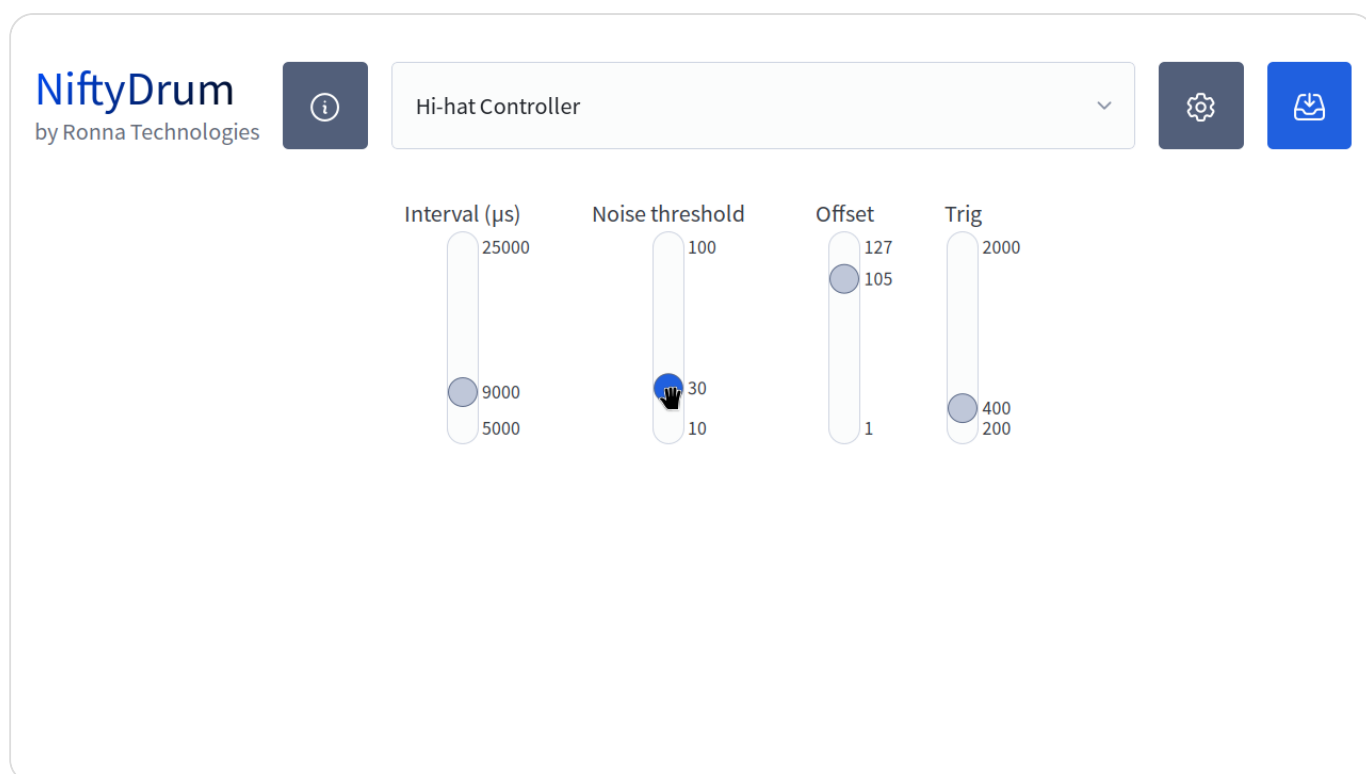
### Hi-Hat Implementation Status

The hi-hat implementation is currently **experimental**. The app settings may evolve in future updates.

### 3.4.1 Parameters

The hi-hat pedal configuration includes four key parameters:

| Parameter                        | Description                                                      |
|----------------------------------|------------------------------------------------------------------|
| <b>Update interval</b>           | Sets the frequency at which the hi-hat pedal updates.            |
| <b>Noise threshold</b>           | Ignores pedal changes below this value to filter out noise.      |
| <b>Pedal offset</b>              | Determines whether the hi-hat is fully closed.                   |
| <b>Velocity threshold (Trig)</b> | Sets the sensitivity threshold for triggering a foot chick note. |



## 3.4.2 CC Message Behavior

- The **Update interval** defines the minimum delay (in microseconds) between two CC messages, typically set to **10ms**.
- The **Noise threshold** sets the minimum change in value required to send a CC message.

## 3.4.3 Foot Chick Note Trigger

The **Pedal offset** and **Velocity threshold (Trig)** determine whether a foot chick note is sent, with the latter acting as a sensitivity threshold.

## 3.5 Firmware Upgrade

The app also supports **firmware upgrades** for your board.

**NiftyDrum**  
by Ronna Technologies



## Advanced settings



### Firmware

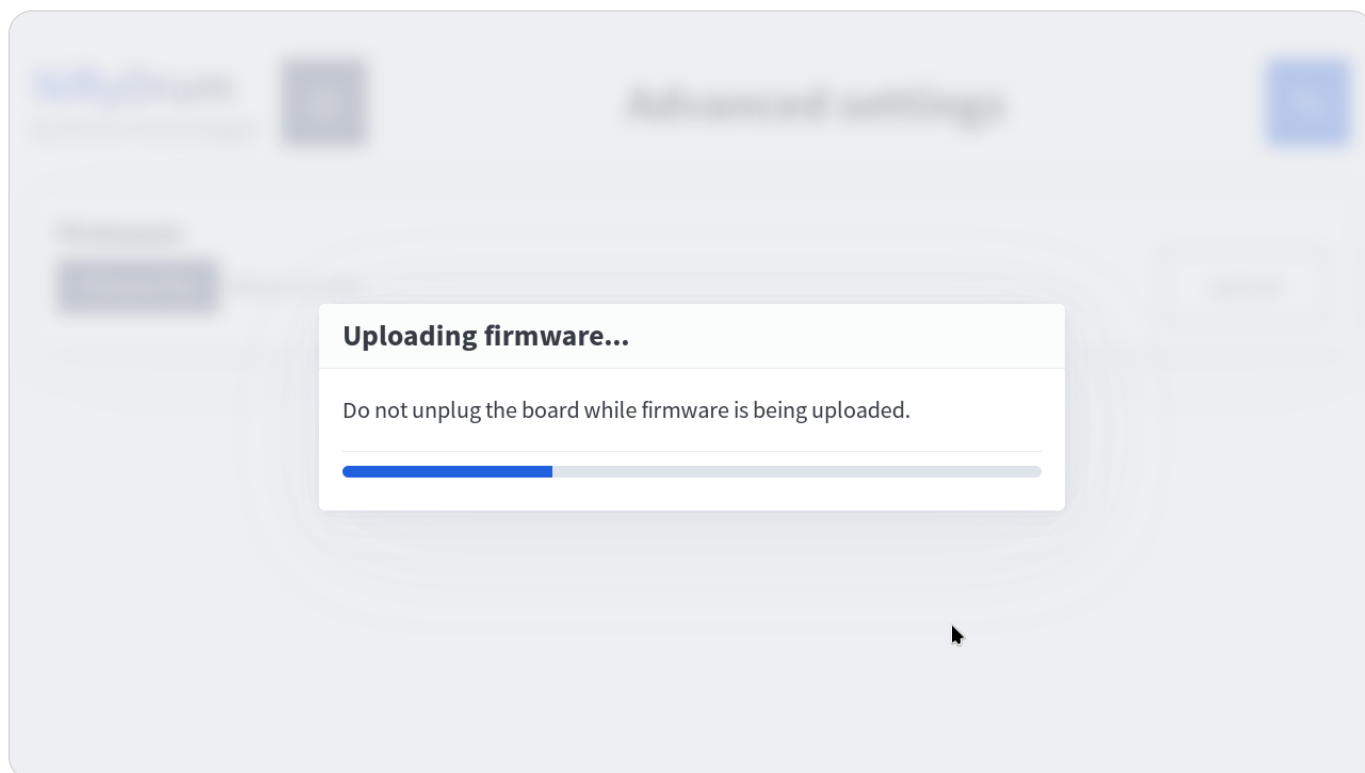
Choose File

NiftyDrum.bin

upload

Here is how to upgrade the firmware:

- From the app's main screen, click the **"gear" icon** to access the **Advanced Settings** screen.
- Select a firmware file (if downloaded as a **.zip** , **unzip it first**).
- Ensure the file has a **.bin** extension and is an **officially supported firmware**.
- Click the **Upload** button and wait for the process to complete.



#### Warning

**Do not unplug the board** during the firmware upload.

#### Troubleshooting

If the firmware upload fails or the board is accidentally unplugged. Download [tkg-flash](#) . **Double-press the reset button** on the board. Upload the firmware via terminal:

Unix

Windows

```
./tkg-flash NiftyDrum.bin
```

```
./tkg-flash.exe NiftyDrum.bin
```

Once the upload is complete, the board will **restart automatically**, and the app will **reconnect** to it.

## 4. Serial Protocol

---

### 4.1 Overview

---

The NiftyDrum board establishes a **virtual serial port over USB** when connected to a computer. It also features a **USART interface** on the Raspberry Pi-compatible GPIO port.

#### ⚠ USART Voltage Limit

The USART pins are **3.3V only**. Exceeding this voltage may damage the board.

### 4.2 Command Syntax

---

All commands begin with a forward slash ( / ). Commands that require arguments follow the format:

```
/command arg1 arg2 arg3
```

#### 4.2.1 Special Commands

---

Special commands do not require parameters. They either modify the board's behavior or provide specific information.

| Command                     | Description                              |
|-----------------------------|------------------------------------------|
| <code>/reset</code>         | Reboots the board in bootloader mode.    |
| <code>/factory_reset</code> | Resets all settings to factory defaults. |
| <code>/sn</code>            | Returns the board's serial number.       |
| <code>/version</code>       | Returns the current firmware version.    |



## 4.2.2 Parameters Commands

The board uses **on-board EEPROM** to persist parameters. The serial protocol provides the following commands to save and load these parameters.

| Command                       | Description                             |
|-------------------------------|-----------------------------------------|
| <code>/load params all</code> | Loads all parameters from EEPROM.       |
| <code>/save params all</code> | Saves all current parameters to EEPROM. |

## 4.2.3 Pad Parameters

Each pad supports **six configurable parameters**:

- Threshold
- Gain
- Scan time
- Mask time
- Decay
- MIDI note

### Instrument Placeholder

The examples below use `snare`. Replace it with `hihat`, `kick`, `crash1`, `tom1`, `tom3`, `ride`, `tom2`, or `crash2` as needed.

## Getters

Use the following commands to **retrieve** pad parameter values:

| Command                           | Description                             | Example Reply |
|-----------------------------------|-----------------------------------------|---------------|
| <code>/get snare threshold</code> | Gets the trigger threshold.             | 15            |
| <code>/get snare gain</code>      | Gets the sensor gain.                   | 1.5           |
| <code>/get snare scan</code>      | Gets the scan time ( $\mu$ s).          | 3000 (3ms)    |
| <code>/get snare mask</code>      | Gets the mask time ( $\mu$ s).          | 20000 (20ms)  |
| <code>/get snare decay</code>     | Gets the decay time ( $\mu$ s).         | 60000 (60ms)  |
| <code>/get snare note</code>      | Gets the MIDI note assigned to the pad. | 36            |

## Setters

Use the following commands to **set** pad parameter values:

| Command                           | Description                     | Example Usage                        |
|-----------------------------------|---------------------------------|--------------------------------------|
| <code>/set snare threshold</code> | Sets the trigger threshold.     | <code>/set snare threshold 15</code> |
| <code>/set snare gain</code>      | Sets the sensor gain.           | <code>/set snare gain 1.5</code>     |
| <code>/set snare scan</code>      | Sets the scan time ( $\mu$ s).  | <code>/set snare scan 3000</code>    |
| <code>/set snare mask</code>      | Sets the mask time ( $\mu$ s).  | <code>/set snare mask 20000</code>   |
| <code>/set snare decay</code>     | Sets the decay time ( $\mu$ s). | <code>/set snare decay 60000</code>  |
| <code>/set snare note</code>      | Sets the MIDI note for the pad. | <code>/set snare note 36</code>      |

**⚠ Value Validation**

The board **does not validate** input values. Incorrect values may cause unexpected behavior.

## 4.3 Hi-Hat Controller Parameters

The hi-hat controller operates with dual functionality:

- It sends CC (Control Change) messages when the pedal is pressed or released slowly.
- It triggers MIDI notes when the pedal movement is fast (e.g., a foot splash or chick).

These two sensors are denoted FSRC for FSR continuous, and FSRT for FSR trigger.

### 4.3.1 Continuous Parameters

| Command                          | Description                                                        | Example Reply |
|----------------------------------|--------------------------------------------------------------------|---------------|
| <code>/get fsrc timeout</code>   | Gets the minimum time interval between two CC messages ( $\mu$ s). | 9000          |
| <code>/get fsrc threshold</code> | Gets FSR noise threshold.                                          | 30            |
| <code>/get fsrc offset</code>    | Gets the value from which the pedal is considered as fully closed. | 105           |

| Command                          | Description                                                        |
|----------------------------------|--------------------------------------------------------------------|
| <code>/set fsrc timeout</code>   | Sets the minimum time interval between two CC messages ( $\mu$ s). |
| <code>/set fsrc threshold</code> | Sets FSR noise threshold.                                          |
| <code>/set fsrc offset</code>    | Sets the value from which the pedal is considered as fully closed. |

## 4.3.2 Trigger Parameters

| Command                          | Description                                                        | Example Reply |
|----------------------------------|--------------------------------------------------------------------|---------------|
| <code>/get fsrt threshold</code> | Gets the minimum time interval between two CC messages ( $\mu$ s). | 9000          |
| <code>/get fsrt scan</code>      | Gets FSR noise threshold.                                          | 30            |
| <code>/get fsrt velocity</code>  | Gets the value from which the pedal is considered as fully closed. | 105           |

| Command                          | Description                                                        |
|----------------------------------|--------------------------------------------------------------------|
| <code>/set fsrt threshold</code> | Sets the minimum time interval between two CC messages ( $\mu$ s). |
| <code>/set fsrt scan</code>      | Sets FSR noise threshold.                                          |
| <code>/set fsrt velocity</code>  | Sets the value from which the pedal is considered as fully closed. |

## 5. Arduino

---

### 5.1 That's a Blue Pill

---

NiftyDrum is powered by the **STM32F103C8T6** microcontroller, the same chip found in the iconic "**Blue Pill**" development board.

If you're unfamiliar with the Blue Pill, it's a widely recognized and versatile board based on the STM32F103C microcontroller family. With **72MHz** clock speed, **64KB of flash** memory, and **20KB of RAM**, it offers plenty of flexibility for customization and experimentation.

### 5.2 Arduino IDE

---

Because NiftyDrum is based on the same microcontroller as the Blue Pill, it is possible to program the board using the Arduino IDE. But for that, you first need to have a C++ compiler that targets the STM32F1.

#### 5.2.1 ARM None-EABI toolchain

---

If you are a Linux user, you can install the **ARM None-EABI toolchain** directly from your package manager. The package is typically named **gcc-arm-none-eabi**. For other platforms, download the official ARM toolchain from [Developer.arm.com](https://developer.arm.com/Tools-Software/GNU-Toolchain) – ARM GNU Toolchain Downloads. Make sure to select the correct version for your operating system.

#### 5.2.2 Arduino STM32 SDK

---

[Arduino STM32 SDK](#)

## 5.2.3 Blink

---

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on
  delay(1000);                     // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off
  delay(1000);                     // wait for a second
}
```